Revised Project Proposal: Schema-Evolving Knowledge Graphs for Insurance

Automated Ontology Induction with Cross-LOB Transfer

Xiaofei Wang, Zechary Chou
CS 584 AI Capstone — Spring 2026
Emory University

February 2026

Abstract

This proposal presents a revised approach to constructing insurance knowledge graphs that automatically induces ontology schemas from raw data and gracefully extends them when new Lines of Business (LOBs) are introduced. Unlike prior work that requires predefined schemas or manual ontology engineering, our pipeline combines techniques from recent advances in LLM-based knowledge graph construction—specifically Open Information Extraction from EDC and community-based clustering from OntoRAG—into a unified framework optimized for the insurance domain. Our key innovation is a **schema evolution mechanism** that measures and maximizes cross-LOB transfer, enabling an ontology trained on flood insurance data to automatically extend when presented with auto insurance data. We target 60%+ schema transfer rate with resource-efficient local LLMs (Llama 3.1 8B, DeepSeek-R1 distilled).

1 Introduction and Motivation

1.1 Problem Context

The insurance industry manages heterogeneous data across multiple Lines of Business (LOBs): property, auto, health, liability, and more. Each LOB has traditionally required separate knowledge engineering efforts to create ontologies that capture domain-specific entities (policies, coverages, exclusions) and events (claims, inspections, payments). This fragmentation leads to:

1. **Redundant engineering**: Similar concepts (e.g., "deductible") are modeled separately per LOB
2. **Inconsistent reasoning**: Cross-LOB analytics require manual schema alignment
3. **Slow adaptation**: New LOBs or regulatory changes require expensive re-engineering

1.2 Limitations of Existing Approaches

Recent work on LLM-based knowledge graph construction has made significant progress, but each approach has limitations for our use case:

Table 1: Limitations of Existing Methods for Insurance KG Construction

Method	Approach	Limitation for Insurance
OntoRAG [1]	Community detection on entity graphs	Fixed to single domain; no LOB transfer
AutoSchemaKG [2]	Conceptualization at web scale	Requires 78,400 GPU hours; impractical
EDC [3]	Open IE → Canonicalize	Relations only; no entity type hierarchy
GraphRAG [4]	Community summaries for retrieval	Assumes pre-existing ontology

1.3 Our Contribution

We propose a **streamlined pipeline** that combines the best elements of recent approaches while introducing a novel **schema evolution mechanism** for cross-LOB transfer:

Your Innovation

No existing paper addresses the problem of automatically extending an ontology when a new Line of Business is introduced. Our system:

1. Performs open information extraction on flood insurance data
2. Directly induces ontology from extracted instances via Leiden clustering (Track A)
3. Applies the same pipeline to auto insurance data (Track B)
4. Measures how much ontology structure transfers vs. requires extension
5. Proposes and validates schema extensions automatically

2 Background: State-of-the-Art Methods

We conducted a systematic review of recent papers on automated ontology/schema induction. This section summarizes the key techniques we adopt and their feasibility for our project.

2.1 OntoRAG: Community-Based Ontology Derivation

OntoRAG [1] transforms unstructured PDFs into queryable ontologies through a six-stage pipeline:

1. Web scraping and PDF parsing (Unstructured library)
2. Hybrid chunking (title-aware + semantic similarity)
3. Information extraction (NER, atomic facts, chunk graphs)
4. Knowledge graph construction via entity clustering
5. **Leiden community detection** for ontology class induction
6. LLM-based property synthesis (Gemini 2.5 Flash)

Key Insight

OntoRAG's use of the Leiden algorithm to partition entity graphs into ontology classes achieves 85% comprehensiveness win rate against vector RAG. We adopt this clustering approach for our Zone 3. The Leiden algorithm is well-implemented in Python libraries (graspologic, igraph) and runs efficiently on graphs with 10K–100K nodes.

The similarity function for clustering key elements k_i and k_j is:

$$\text{sim}(k_i, k_j) = \begin{cases} 1 & \text{if } \cos(\vec{v}_{\text{name}}(k_i), \vec{v}_{\text{name}}(k_j)) \geq \theta_{\text{name}} \\ & \text{and } \cos(\vec{v}_{\text{def}}(k_i), \vec{v}_{\text{def}}(k_j)) \geq \theta_{\text{def}} \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

Feasibility: The Unstructured library handles PDFs reliably, and Leiden clustering scales to our dataset size. We can use all-MiniLM-L6-v2 embeddings for cosine similarity, which runs locally without API costs.

2.2 AutoSchemaKG: Conceptualization Without Predefined Schema

AutoSchemaKG [2] demonstrates large-scale knowledge graph construction without predefined schemas:

1. Input processing: filter, segment, batch documents
2. Triple extraction: LLM extracts entities *and events*
3. Conceptualization: LLM assigns semantic categories
4. Schema induction: cluster concepts into types
5. KG assembly: integrate triples with induced schema

Key Insight

AutoSchemaKG's dual modeling of **entities** (static things) and **events** (dynamic occurrences) is critical for insurance. Policies and Coverages are entities; Claims and Payments are events. We adopt this distinction in our Zone 2 classification step. Additionally, AutoSchemaKG demonstrates that schema-free extraction followed by clustering can produce high-quality ontologies—we apply this principle by combining extraction with Leiden clustering.

The ATLAS knowledge graph constructed by AutoSchemaKG achieves 92% semantic alignment with human-crafted schemas—with zero manual intervention.

Feasibility: While AutoSchemaKG used web-scale data (78,400 GPU hours), we scale this down to insurance-specific corpora. The key insight we adopt is the entity/event distinction and the principle of schema-free extraction. We use Leiden clustering (from OntoRAG) instead of AutoSchemaKG's conceptualization pipeline, which is more efficient for our dataset size.

2.3 EDC: Extract-Define-Canonicalize

EDC [3] addresses the problem of scaling to large schemas by decoupling extraction from schema alignment:

1. **Extract**: Open information extraction (no schema constraints)

2. **Define**: LLM generates definitions for extracted relation types
3. **Canonicalize**: Map to target schema via embedding similarity

When no target schema exists, EDC performs **self-canonicalization**: clustering similar relations and generating canonical definitions.

Key Insight

EDC's three-phase separation allows us to first extract freely, then impose structure. This is more flexible than methods that require schema in the prompt (which fails for large schemas exceeding context length).

Feasibility: EDC's approach is highly practical. Open IE with local LLMs produces reasonable triples (we validated with Llama 3.1 8B on sample insurance text). The canonicalization step uses vector similarity, which is fast and deterministic.

3 Proposed Architecture

3.1 System Overview

Our pipeline consists of four zones that implement a bottom-up ontology induction approach. Data flows through progressive abstraction stages: raw text → entity mentions → resolved entities → entity types → ontology classes → hierarchical schema.

Bottom-Up Abstraction Stages:

1. **Zone 1**: Raw documents → Structured chunks
2. **Zone 2a**: Chunks → Entity mentions + Relations (triples)
3. **Zone 2b**: Entity mentions → Entity/Event classification
4. **Zone 2c**: Classified mentions → Resolved entities with provisional types
5. **Zone 3a**: Entity types → Ontology classes (via clustering) → Hierarchy
6. **Zone 3b**: Raw ontology → Validated and refined ontology
7. **Zone 3c**: Single-LOB ontology → Multi-LOB extended ontology
8. **Zone 4**: Abstract schema + Concrete instances → Queryable knowledge graph

Key Characteristic: No predefined schema is provided at any stage. The ontology emerges purely from data through clustering and LLM-based abstraction.

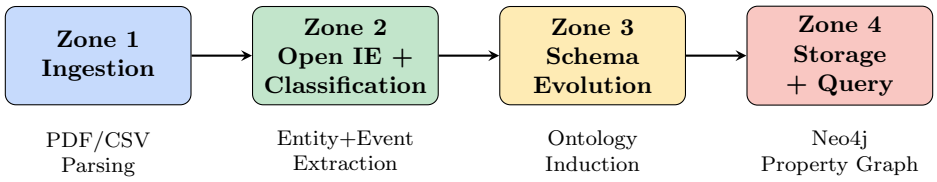

Figure 1: High-level architecture showing the four processing zones

3.2 Zone 1: Multimodal Ingestion

Zone 1 handles heterogeneous input formats and converts them into a unified chunk representation suitable for downstream LLM processing.

3.2.1 Input Data Types

Structured Data (NFIP CSVs): Policy and claims tables contain approximately 30–50 columns each. Each row is converted into a text chunk that preserves the semantic context of the record. For example, a policy row becomes: "Policy XYZ has effective date 2024-01-01, coverage limit $250,000, deductible $1,000, located at property ABC in flood zone X."

Unstructured Data (SFIP PDFs): Standard Flood Insurance Policy documents are 30–50 page legal texts with sections, subsections, and embedded tables. We use the Unstructured library's hierarchical detection to identify:

- Section headers (e.g., "II. COVERAGE")
- Subsections (e.g., "A. Building Property")
- List items (conditions, exclusions)
- Tables (deductible schedules, coverage limits)

Tables in PDFs are extracted with PyMuPDF and converted to markdown format for better LLM comprehension.

Semi-Structured Data (Kaggle CSVs): Auto insurance datasets have different schemas than flood insurance. We apply column name normalization and type detection to identify common concepts (policy IDs, dates, amounts) even when column names differ.

3.2.2 Hybrid Chunking Strategy

Following OntoRAG's proven approach, we implement a two-stage chunking process:

Stage 1 - Title-Based Chunking: Split documents at structural boundaries (section headers, subsection headers). Each chunk must have minimum 100 tokens and maximum 800 tokens. If a section is too long, subdivide at paragraph boundaries.

Stage 2 - Semantic Merging: Adjacent chunks are merged if their embedding cosine similarity exceeds threshold $\tau_{\text{merge}} = 0.85$. This prevents splitting semantically cohesive content (e.g., a multi-paragraph explanation of a single coverage type).

3.2.3 Metadata Preservation

Each chunk carries metadata:

- Source document ID
- Document type (policy, claim, SFIP clause)
- Section hierarchy (for PDFs)
- Record identifiers (for CSVs)
- Temporal information (effective dates, claim dates)

This metadata enables later graph construction to link entities across sources (e.g., linking a claim record to the policy record and relevant SFIP clauses).

3.3 Zone 2: Open IE + Entity/Event Classification

Zone 2 performs information extraction without schema constraints and classifies extracted elements as entities or events. This creates a pool of entity mentions that will be resolved and typed in subsequent steps.

3.3.1 Step 2a: Open Information Extraction

The extraction phase uses Llama 3.1 8B in a few-shot learning configuration. No predefined schema is provided—the LLM extracts any factual relationships it observes.

Prompt Structure: The prompt includes 3–5 insurance-specific examples showing desired triple format. Examples cover:

- Policy-coverage relationships
- Claim-loss event relationships
- Temporal relationships (policy issuance, claim filing)
- Causal relationships (exclusion → claim denial)

Output Format: Structured JSON with fields:

- `subject`: Entity or event instance (string identifier)
- `subject_context`: Full text describing the subject
- `relation`: Relationship type (free text)
- `object`: Target entity/event/attribute
- `object_context`: Full text describing the object
- `confidence`: Model confidence (0.0–1.0)
- `source_chunk_id`: Source chunk ID for provenance

Example Output:

- Subject: "Policy 5123456789"
- Subject context: "The flood insurance policy with policy number 5123456789 issued on 2024-01-15"
- Relation: "has_coverage_limit"
- Object: "$250,000"

Quality Control: We filter triples with confidence < 0.7 and apply validation rules:

- Subject and object must be non-empty
- Relations must be specific enough (not just "is", "has", "relates to")
- No circular references within same triple

Examples of Good vs. Bad Triples:

Table 2: Triple Quality Examples for Open IE

Quality	Triple	Reason
GOOD	(Policy 5123456789, has_coverage_limit, $250,000)	Specific relation, concrete values
GOOD	(Policy 5123456789, located_at, 123 Oak St)	Clear relationship
GOOD	(Claim C-987654, filed_for, flood damage)	Action verb, specific event
BAD	(Policy, has, coverage)	Too generic, no concrete instances
BAD	(Policy XYZ, relates to, something)	Vague relation and object
BAD	(Claim, happened, 2024)	Incomplete - which claim?
BAD	(Deductible, applies,)	Missing object
MARGINAL	(Policy ABC, has, $250K limit)	"has" is generic, but object is informative. **Fix**: Extract as (Policy ABC, has_coverage_limit, $250K)
MARGINAL	(Policy covers flood damage)	Relation embedded in text. **Fix**: Extract as (Policy XYZ, provides_coverage_for, flood damage)

Automated Quality Filters:

1. **Relation specificity**: Reject if relation is in blocklist {"is", "has", "contains", "relates_to", "associated_with"}
2. **Entity concreteness**: Subject should contain identifier (number, name) or be from controlled vocabulary
3. **Completeness**: All three components (subject, relation, object) must be non-empty after parsing
4. **Redundancy**: If extracting from structured data (CSV), prefer column-based relations over generic "has"

Few-Shot Prompting Strategy:
Include 5 positive and 3 negative examples in the prompt:
Positive examples:

```
[GOOD] (Policy 5123456789, effective_date, 2024-01-15)
[GOOD] (Claim C-789012, filed_against, Policy 5123456789)
[GOOD] (Property at 456 Elm St, located_in_flood_zone, AE)
[GOOD] (Policy 5123456789, has_deductible, $1,000)
[GOOD] (Claim C-789012, loss_occurred_on, 2024-03-20)
```

Negative examples (what NOT to extract):

```
[BAD] (Policy, has, something) -- too generic
[BAD] (It, relates to, them) -- pronouns without referents
[BAD] (The property, is, there) -- vague
```

3.3.2 Step 2b: Entity vs. Event Classification

A lightweight classifier (rule-based + LLM fallback) distinguishes static entities from dynamic events. This is critical for insurance because entities and events have different property types and relationship semantics.

Rule-Based Classification:

- Keywords indicating events: "claim", "payment", "loss", "inspection", "issuance"
- Temporal markers: dates, durations, "occurred", "filed"
- Action verbs in relation types: "filed", "paid", "inspected"

LLM Fallback: For ambiguous cases, query Llama 3.1 8B with examples:

- Entity examples: "policy contract", "coverage limit", "property address"
- Event examples: "claim submission", "damage assessment", "premium payment"

Output: Each extracted element is tagged as ENTITY or EVENT, with provisional subtype (POLICY, CLAIM, COVERAGE for entities; FILING, PAYMENT, INSPECTION for events). Note that these provisional subtypes are initial guesses and will be refined through clustering.

3.3.3 Step 2c: Entity Resolution and Typing

Before clustering, we must resolve entity mentions (identify when different strings refer to the same entity) and extract entity types.

Entity Resolution (Coreference):

We use a two-stage approach:

Stage 1 - String Matching:

- Exact match: "Policy 5123456789" = "Policy 5123456789" $\rightarrow$ merge
- Fuzzy match: "Policy 5123456789" $\approx$ "policy #5123456789" $\rightarrow$ merge if edit distance < 3
- Canonical IDs: Extract policy numbers, claim numbers, addresses as identifiers

Stage 2 - Embedding Similarity:

For mentions without canonical IDs, use embedding similarity:

$$\text{same_entity}(m_i, m_j) = \begin{cases} \text{TRUE} & \text{if } \cos(\vec{v}(m_i), \vec{v}(m_j)) > 0.92 \\ \text{LLM_VERIFY} & \text{if } 0.85 \leq \cos(\vec{v}(m_i), \vec{v}(m_j)) \leq 0.92 \\ \text{FALSE} & \text{if } \cos(\vec{v}(m_i), \vec{v}(m_j)) < 0.85 \end{cases} \tag{2}$$

where $\vec{v}(m_i)$ is the embedding of the entity context (not just the name).

Entity Typing:

After resolution, each unique entity has:

- **Canonical ID**: Resolved identifier (e.g., "Entity_Policy_5123456789")
- **Mentions**: All text spans referring to this entity
- **Attributes**: Properties extracted from triples (coverage limit, effective date, etc.)
- **Relations**: Links to other entities
- **Embedding**: Average embedding of all mention contexts

We then extract a **provisional type** for each entity using an LLM prompt:

Prompt: "Given this entity with mentions: [list of mentions] and attributes: [list of attributes], what is the most specific type? Output format: {"type": "PolicyDocument", "confidence": 0.9}"

Output from Zone 2: A set of typed entities $E = \{e_1, e_2, \ldots, e_n\}$ where each entity has:

- Canonical ID
- Provisional type (e.g., "flood_policy", "property_damage_claim")
- Embedding vector
- Attributes (key-value pairs)
- Relations to other entities

3.4 Zone 3: Schema Evolution (Our Innovation)

Zone 3 is the core contribution of this project. It induces an ontology structure from conceptualized elements and provides mechanisms to evolve that ontology when new data sources (LOBs) are introduced.

3.4.1 Component 3a: Ontology Induction via Leiden Clustering

The goal is to perform bottom-up ontology induction: cluster similar entity types into ontology classes, then infer hierarchical relationships.

Bottom-Up Stages:

1. **Input**: Typed entities from Zone 2c (each with provisional type like "flood_policy", "property_claim")
2. **Type Aggregation**: Group entities by provisional type, compute type-level statistics
3. **Type Clustering**: Cluster similar types into ontology classes
4. **Hierarchy Induction**: Discover subclass relationships between clusters

Step 1: Type Aggregation
From n entities, we extract k unique provisional types. For each type t:

- **Type embedding**: Average embedding of all entities with this type
- **Instance count**: Number of entities with this type
- **Attributes**: Union of all attributes across entities (e.g., all "flood_policy" entities have {coverage_limit, effective_date, ...})
- **Relation signature**: Set of relation types this entity type participates in

Step 2: Type Similarity Graph Construction
We build a weighted graph where:

- **Nodes**: Unique provisional types $T = \{t_1, t_2, \ldots, t_k\}$
- **Edges**: Weighted by combined semantic and relational similarity

The edge weight between types t_i and t_j is:

$$w_{ij} = \alpha \cdot \text{sem}(t_i, t_j) + \beta \cdot \text{cooccur}(t_i, t_j) + \delta \cdot \text{rel_sim}(t_i, t_j) \tag{3}$$

where $\alpha + \beta + \delta = 1$ and we use $\alpha = 0.5, \beta = 0.3, \delta = 0.2$.

Component Definitions:

Semantic similarity:

$$\text{sem}(t_i, t_j) = \cos(\vec{v}(t_i), \vec{v}(t_j)) \tag{4}$$

where $\vec{v}(t_i)$ is the type embedding (average of entity embeddings with type t_i).

Co-occurrence:

$$\text{cooccur}(t_i, t_j) = \frac{\#\text{ triples where entity of type } t_i \text{ relates to entity of type } t_j}{\#\text{ triples involving } t_i \text{ or } t_j} \tag{5}$$

Example: If "flood_policy" entities appear in 100 triples total, "property_address" entities in 80 triples, and 30 triples connect them (e.g., "policy X located_at address Y"), then:

$$\text{cooccur}(\text{flood_policy}, \text{property_address}) = \frac{30}{100 + 80 - 30} = 0.2$$

Relational similarity: Measures how similar the relation signatures are.

$$\text{rel_sim}(t_i, t_j) = \frac{|R(t_i) \cap R(t_j)|}{|R(t_i) \cup R(t_j)|} \tag{6}$$

where $R(t_i)$ is the set of relation types that entities of type t_i participate in.

Example: If "flood_policy" has relations {has_coverage, located_at, issued_to} and "auto_policy" has {has_coverage, located_at, covers_vehicle}, then:

$$\text{rel_sim} = \frac{|\{\text{has_coverage}, \text{located_at}\}|}{|\{\text{has_coverage}, \text{located_at}, \text{issued_to}, \text{covers_vehicle}\}|} = \frac{2}{4} = 0.5$$

Note on Relation Canonicalization: Before computing $R(t_i)$, we group similar relation types to reduce noise:

- Cluster relations by string similarity (edit distance < 3) and embedding similarity (> 0.85)
- Example: {has_coverage, provides_coverage, includes_coverage} $\rightarrow$ canonical "has_coverage"
- This prevents relation signature inflation from synonymous relation types

Thresholding: Set $w_{ij} \leftarrow 0$ if $w_{ij} < \tau = 0.3$ to remove weak connections and improve cluster quality.

Step 3: Leiden Clustering on Types

Apply the Leiden community detection algorithm:

- **Input**: Weighted graph of types
- **Resolution parameter** γ: Controls granularity
  - Lower γ (e.g., 0.5): Broader classes ("Insurance Product")
  - Higher γ (e.g., 1.5): Finer classes ("Flood Policy", "Auto Policy")
- **Tuning**: We iterate over $\gamma \in \{0.5, 0.75, 1.0, 1.25, 1.5\}$
- **Selection**: Choose γ that maximizes modularity Q

Modularity score:

$$Q = \frac{1}{2m} \sum_{ij} \left[w_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \tag{7}$$

where m is total edge weight, k_i is sum of edge weights for node i, and $\delta(c_i, c_j) = 1$ if nodes i and j are in the same cluster.

Output: Partition of types into clusters $C = \{C_1, C_2, \ldots, C_p\}$.

Example result:

- C_1: {flood_policy, wind_policy, earthquake_policy} $\rightarrow$ "PropertyPolicy"
- C_2: {auto_collision_coverage, auto_liability_coverage} $\rightarrow$ "AutoCoverage"
- C_3: {flood_claim, property_damage_claim} $\rightarrow$ "PropertyClaim"

Step 4: Ontology Class Naming

For each cluster C_i, use LLM to generate:

LLM Prompt:

```
Given these entity types and their attributes:
- flood_policy: {coverage_limit, effective_date, premium, deductible}
- wind_policy: {coverage_limit, effective_date, premium, max_wind_speed}
- earthquake_policy: {coverage_limit, effective_date, premium, seismic_zone}

Generate an abstract ontology class name that represents all these types.

Output format:
{
  "class_name": "PropertyInsurancePolicy",
  "definition": "A policy that provides coverage for property damage",
  "common_properties": ["coverage_limit", "effective_date", "premium"],
  "discriminative_properties": ["deductible", "max_wind_speed", "seismic_zone"]
}
```

Step 5: Hierarchy Induction

Discover subclass relationships using two methods:

Method 1 - Sub-Community Detection:

For each large cluster ($|C_i| > 10$ types), run Leiden again with higher resolution $\gamma' = 2\gamma$ to find sub-communities. These become subclasses.

Example:

- Cluster: {flood_policy, wind_policy, earthquake_policy, flood_claim, wind_claim}
- Sub-clusters at γ':
  - Sub-cluster 1: {flood_policy, wind_policy, earthquake_policy} → "PropertyPolicy"
  - Sub-cluster 2: {flood_claim, wind_claim} → "PropertyClaim"
- Relationship: Both are types of parent cluster "PropertyInsuranceArtifact"

Method 2 - Relation-Based Inference:

If types t_i and t_j have asymmetric relationships, check for subclass relation:

- If $R(t_i) \subseteq R(t_j)$ and $\text{sem}(t_i, t_j) > 0.7$, then t_i may be subclass of t_j
- Use LLM to verify: "Is 'flood_policy' a type of 'property_policy'?"

Hierarchy Construction:

Build a DAG (Directed Acyclic Graph) of subclass relationships:

- Nodes: Ontology classes from clustering
- Edges: $C_i \to C_j$ if C_i is subclass of C_j
- Depth limit: 3 levels to prevent over-fragmentation
- Root: "InsuranceEntity" or "InsuranceEvent"

Step 6: Property Synthesis

For each ontology class C_i containing types $\{t_1, t_2, \ldots, t_m\}$:

Property Sources:

Properties come from two sources with different extraction methods:

Source 1 - Structured Data (CSVs):

- Column names directly become property names
- Example: CSV column "CoverageLimit" → property "coverage_limit"
- Data types inferred from column values

Source 2 - Unstructured Data (PDFs):

- Extract attribute-value pairs from text using LLM
- Prompt: "Extract structured properties from: [entity context text]. Format: {property: value}"
- Example: "Policy XYZ provides coverage up to $250,000" → {"coverage_limit": 250000}
- Filter noisy extractions: Require property to appear in $\geq 40\%$ of entities of the same type (minimum 2 instances). This adaptive threshold ensures rare types don't lose valid properties.

Aggregate properties from all member types:

1. Collect all attributes: $A = \bigcup_{t \in C_i} \text{attributes}(t)$
2. Cluster similar property names via string similarity and embeddings
   - Example: "limit", "coverage_limit", "max_amount" → "coverageLimit"
   - Use threshold: normalized edit distance < 0.3 OR embedding similarity > 0.85
3. Compute property frequency:
   - **Required**: appears in $> 80\%$ of types
   - **Optional**: appears in $< 80\%$ of types
4. Infer property types from values:
   - Numeric: "coverage_limit" → float (if values like 250000, 350000)
   - Temporal: "effective_date" → date (if values like "2024-01-15")
   - Categorical: "flood_zone" → enum {A, AE, X, ...} (if values from fixed set)
   - Text: "description" → string (fallback)

Output from Zone 3a: A hierarchical ontology $\mathcal{O}$ with:

- **Classes**: $C = \{C_1, C_2, \ldots, C_p\}$ with names, definitions
- **Hierarchy**: DAG of subclass relationships
- **Properties**: For each class, required and optional properties with types
- **Type-to-Class mapping**: Which provisional types belong to which ontology class
- **Instance-to-Type mapping**: Which entities have which provisional type (carried from Zone 2)

3.4.2 Component 3b: Ontology Validation and Refinement

After inducing the ontology, we validate the quality of classes and hierarchies before finalizing the schema.

Validation Checks:

1. Class Coherence Check:

For each ontology class C_i, verify that member types are actually similar:

- Compute pairwise similarity between all types in C_i
- If average similarity < 0.6, the cluster may be incoherent

- Use LLM to verify: "Do these types belong together: {flood_policy, wind_policy, earthquake_policy}?"

2. Hierarchy Consistency Check:
Verify that subclass relationships are valid:

- For $C_i \subseteq C_j$, check that properties of C_i are superset of C_j's properties
- Check for cycles in the DAG (should be impossible, but validate)
- Check for redundant paths: if $C_i \subseteq C_j$ and $C_i \subseteq C_k$ and $C_k \subseteq C_j$, remove redundant edge $C_i \to C_j$

3. Property Conflict Resolution:
When aggregating properties from member types, conflicts may arise:

- **Type conflicts**: Same property name but different types (e.g., "limit" as string vs. float)
  - Resolution: Use majority vote or most specific type (float ¿ int ¿ string)
- **Required vs. optional**: Some types have property, others don't
  - Resolution: Mark as optional if $< 80\%$ frequency

4. Ontology Class Naming Conflicts:
Check if LLM-generated class names are ambiguous:

- If two classes have same name or very similar names (edit distance < 3), ask LLM to disambiguate
- Example: "PropertyPolicy" and "PropertyInsurancePolicy" $\to$ choose one or rename

5. Coverage Check:
Verify that all provisional types from Zone 2c are mapped to some ontology class:

- If a type is orphaned (not in any cluster), either:
  - Re-cluster with lower threshold, or
  - Create singleton class for this type

Refinement Process:
For classes that fail validation:
Option 1 - Split: If coherence is low, split the cluster by running Leiden at higher γ
Option 2 - Merge: If two classes are very similar (similarity > 0.9), merge them
Option 3 - Manual Review: For ambiguous cases, flag for human review (store in review queue)
Feedback Loop:
Validation results are logged and used to tune clustering parameters:

- If many classes fail coherence check $\to$ increase γ (finer granularity)
- If too many singleton classes $\to$ decrease γ (coarser granularity)
- If hierarchy is too deep (¿ 3 levels) $\to$ reduce sub-clustering iterations

Output: A validated and refined ontology $\mathcal{O}_{\text{validated}}$ ready for storage and extension testing.

3.4.3 Component 3c: Schema Extension Mechanism

This is our primary innovation. When processing a new LOB (e.g., auto insurance after flood insurance), ontology classes from the new LOB may not align with the existing ontology. We propose three extension types with precise matching criteria.

Setup:

- $\mathcal{O}_A$: Ontology induced from LOB A (flood insurance) with classes $\{C_1^A, C_2^A, \ldots\}$
- $\mathcal{O}_B$: Ontology induced independently from LOB B (auto insurance) with classes $\{C_1^B, C_2^B, \ldots\}$
- Goal: Measure how much of $\mathcal{O}_A$ can be reused for LOB B

Key Insight

Our evaluation follows a two-experiment design:

Experiment 1 - Transfer Measurement (Primary):

1. Cluster flood data independently using Zone 3a $\rightarrow \mathcal{O}_A$
2. Cluster auto data independently using Zone 3a (same γ, thresholds) $\rightarrow \mathcal{O}_B$
3. Compare $\mathcal{O}_A$ and $\mathcal{O}_B$ class-by-class to compute CTR, PTR
4. Result: Quantified transfer metrics answering "How much of flood ontology applies to auto?"

Experiment 2 - Practical Extension (Secondary):

1. Start with $\mathcal{O}_A$ (flood ontology as base)
2. For each class in $\mathcal{O}_B$, integrate into $\mathcal{O}_A$ as subclass/sibling/new branch
3. Result: $\mathcal{O}_A^{\text{extended}}$ unified multi-LOB ontology
4. Deliverable: Single knowledge graph covering both insurance domains

Experiment 1 measures theoretical transfer. Experiment 2 demonstrates practical utility.

Class Matching Criteria:

For each class C_i^B from auto insurance, find the best match in $\mathcal{O}_A$:

$$\text{best_match}(C_i^B) = \arg\max_{C_j^A \in \mathcal{O}_A} \text{match_score}(C_i^B, C_j^A) \tag{8}$$

The match score combines three components:

$$\text{match_score}(C_i^B, C_j^A) = 0.5 \cdot \text{sem}(C_i^B, C_j^A) + 0.3 \cdot \text{prop}(C_i^B, C_j^A) + 0.2 \cdot \text{struct}(C_i^B, C_j^A) \tag{9}$$

Component 1 - Semantic similarity:

$$\text{sem}(C_i^B, C_j^A) = \frac{\cos(\vec{v}(C_i^B), \vec{v}(C_j^A)) + \text{name_sim}(C_i^B, C_j^A)}{2} \tag{10}$$

where $\vec{v}(C)$ is the class embedding (average of type embeddings in the class) and name_sim is normalized edit distance between class names.

Component 2 - Property overlap:

$$\text{prop}(C_i^B, C_j^A) = \frac{|P(C_i^B) \cap P(C_j^A)|}{|P(C_i^B) \cup P(C_j^A)|} \tag{11}$$

where $P(C)$ is the set of property names for class C.
Component 3 - Structural similarity:

$$\text{struct}(C_i^B, C_j^A) = \frac{|R(C_i^B) \cap R(C_j^A)|}{|R(C_i^B) \cup R(C_j^A)|} \tag{12}$$

where $R(C)$ is the set of relation types that entities of this class participate in.
Extension Classification:
Based on the best match score $s_{\text{best}} = \text{match_score}(C_i^B, C_j^A)$, classify the extension type:

$$\text{ExtensionType}(C_i^B) = \begin{cases} \text{DIRECT_REUSE} & \text{if } s_{\text{best}} > 0.90 \\ \text{SUBCLASS} & \text{if } 0.75 < s_{\text{best}} \leq 0.90 \text{ and LLM confirms subclass} \\ \text{SIBLING} & \text{if } 0.60 < s_{\text{best}} \leq 0.75 \text{ and LLM confirms sibling} \\ \text{NEW_BRANCH} & \text{if } s_{\text{best}} \leq 0.60 \end{cases} \tag{13}$$

Extension Type Details:
Type 1 - Direct Reuse ($s > 0.90$):

- The class from LOB B directly maps to existing class from LOB A
- Example: "AutoPolicy" (from auto) matches "InsurancePolicy" (from flood) at 0.93
- Action: Map types from C_i^B to C_j^A without modification

Type 2 - Subclass Extension ($0.75 < s \leq 0.90$):

- The class from LOB B is a specialization of a class from LOB A
- Example: "CollisionCoverage" (from auto) is subclass of "Coverage" (from flood) at 0.82
- Action: Add C_i^B as subclass: $C_i^B \subseteq C_j^A$
- LLM verification prompt: "Is [auto class name and properties] a specific type of [flood class name and properties]?"

Type 3 - Sibling Extension ($0.60 < s \leq 0.75$):

- The class from LOB B is parallel to a class from LOB A under a common parent
- Example: "Vehicle" (from auto) is sibling to "Property" (from flood), both under "InsurableAsset"
- Action: Find or create common parent class, add C_i^B as sibling
- LLM verification prompt: "What is the common abstraction for [flood class] and [auto class]?"

Type 4 - New Branch ($s \leq 0.60$):

- The class from LOB B requires a completely new top-level class
- Example: "Driver" (from auto) has no analog in flood insurance
- Action: Add C_i^B as new top-level class in $\mathcal{O}_A$

Transfer Rate Metrics:
We define multiple transfer metrics to capture different aspects:
Metric 1 - Class Transfer Rate:

$$\text{CTR} = \frac{|\{\text{DIRECT_REUSE}\}| + |\{\text{SUBCLASS}\}|}{|C^B|} \times 100\% \tag{14}$$

This measures what percentage of classes from LOB B either directly reuse or minimally extend (via subclass) the LOB A ontology. **Target: CTR** $> 60\%$

Metric 2 - Property Transfer Rate:

$$\text{PTR} = \frac{\sum_{C_i^B} |P(C_i^B) \cap P(\text{best_match}(C_i^B))|}{\sum_{C_i^B} |P(C_i^B)|} \times 100\% \tag{15}$$

This measures what percentage of properties in LOB B already exist in LOB A's matched classes. **Target: PTR** $> 50\%$

Extension Process:

For each class C_i^B in the auto insurance ontology:

1. Compute match scores with all classes in $\mathcal{O}_A$
2. Find best match $C_j^A = \text{best_match}(C_i^B)$
3. Classify extension type based on s_{best}
4. If SUBCLASS or SIBLING, query LLM for verification:

```
Flood class: {name: "Coverage", properties: ["limit", "deductible"],
              definition: "..."}
Auto class: {name: "CollisionCoverage",
             properties: ["limit", "deductible", "collision_type"],
             definition: "..."}

Is the auto class a subclass of the flood class?
Options: YES/NO/SIBLING/NEW_BRANCH
Reasoning: [explain]
```

5. Apply extension to $\mathcal{O}_A$
6. Log extension for metric calculation

Controlled Experiment:

To validate our approach, we compare:

- **Baseline**: $\mathcal{O}_B^{\text{fresh}}$ - ontology induced from auto data alone (same γ parameters)
- **Transfer**: $\mathcal{O}_A^{\text{extended}}$ - flood ontology extended with auto data

Success criteria:

- CTR $> 60\%$: Majority of auto classes map to flood ontology
- PTR $> 50\%$: Majority of properties are reusable
- Extension quality: $> 80\%$ of LLM-verified extensions are judged correct by human evaluator

Output: Extended ontology $\mathcal{O}_A^{\text{extended}}$ with logged transfer metrics and extension decisions for analysis.

3.5 Zone 4: Storage and Query Interface

Zone 4 materializes both the induced ontology and extracted entity instances in Neo4j property graph database.

3.5.1 Schema Design Philosophy

We use Neo4j property graphs instead of RDF/OWL for pragmatic reasons:

- Property graphs natively support properties on relationships (e.g., claim-policy link with timestamp)
- Cypher query language is more intuitive than SPARQL for graph traversal
- Neo4j provides excellent visualization tools (Neo4j Bloom)
- No need for reasoning engine—our ontology is lightweight

3.5.2 Node and Relationship Design

We store THREE layers in the same graph:

Layer 1 - Entity Instances (the actual data):

- Each extracted entity becomes a node with dynamic label based on its type
- Example: `(:FloodPolicy {policy_number: "5123456789", effective_date: "2024-01-15", coverage_limit: 250000})`
- Metadata properties: `source_doc_id`, `extraction_confidence`, `created_at`
- Estimated count: 100K–150K instance nodes (from 100K policy records + 50K claims + extracted entities from PDFs)

Layer 2 - Entity Types (provisional types from Zone 2c):

- Each provisional type becomes a node: `(:EntityType {name: "flood_policy", instance_count: 1234})`
- Links to instances: `(instance)-[:HAS_TYPE]->(type)`
- Estimated count: 50–200 type nodes

Layer 3 - Ontology Classes (schema from Zone 3a):

- Each ontology class: `(:OntologyClass {name: "PropertyPolicy", level: 1, definition: "...", properties: ["coverage_limit", "effective_date", ...]})`
- Hierarchy links: `(subclass)-[:SUBCLASS_OF]->(superclass)`
- Type-to-Class mapping: `(type)-[:INSTANCE_OF_CLASS]->(class)`
- Estimated count: 20–50 ontology class nodes

Relationship Types Between Instances:
Extracted relations become typed relationships between entity instances:

- `(claim)-[:FILED_AGAINST]->(policy)`
- `(policy)-[:HAS_COVERAGE]->(coverage)`
- `(claim)-[:INVOLVES_EXCLUSION]->(exclusion)`
- `(policy)-[:LOCATED_AT]->(property)`
- `(claim)-[:RESULTED_IN_PAYMENT]->(payment)`

Complete Schema Diagram:

```
+------------------+
| OntologyClass    |  (20-50 nodes)
| - name           |
```

```
| - definition     |
| - properties[]   |
+--------+---------+
         |
         | [:SUBCLASS_OF]
         |
         v
+------------------+
| EntityType       |  (50-200 nodes)
| - name           |
| - instance_count |
+--------+---------+
         |
         | [:INSTANCE_OF_CLASS]
         |
         v
+------------------+
| Entity Instance  |  (100K-150K nodes)
| Label: FloodPolicy, AutoPolicy, Claim, Coverage, etc.
| - id             |
| - properties...  |
+------------------+
         |
         | [:FILED_AGAINST], [:HAS_COVERAGE], etc.
         |
         v
    [other instances]
```

Example Concrete Instance:

```
// Ontology layer
(:OntologyClass {name: "PropertyPolicy", level: 1})
  ^ [:SUBCLASS_OF]
(:OntologyClass {name: "InsurancePolicy", level: 0})

// Type layer
(:EntityType {name: "flood_policy", instance_count: 45678})
  ^ [:INSTANCE_OF_CLASS]
(:OntologyClass {name: "PropertyPolicy"})

// Instance layer
(:FloodPolicy {
  policy_number: "5123456789",
  effective_date: "2024-01-15",
  coverage_limit: 250000,
  deductible: 1000,
  flood_zone: "AE"
})
```

```
  ^ [:HAS_TYPE]
(:EntityType {name: "flood_policy"})

// Relations between instances
(:Claim {claim_id: "C-789012", date_of_loss: "2024-03-20"})
  -[:FILED_AGAINST]->
(:FloodPolicy {policy_number: "5123456789"})
```

Why Store All Three Layers?

1. **Instances**: Needed for actual queries ("Show me all claims for policy X")
2. **Types**: Enable type-level analytics ("How many types did we discover?")
3. **Classes**: Support schema queries ("What is the ontology hierarchy?") and enable schema evolution tracking

Estimated Total Graph Size:

- Nodes: ∼100K–150K (mostly instances)
- Relationships: ∼200K–300K (instance-to-instance + type/class links)
- This is well within Neo4j Community Edition limits

3.5.3 Query Optimization Strategy

To ensure efficient queries on the 100K+ node graph, we implement the following optimizations:

Indexes on Key Properties:

- `CREATE INDEX ON :FloodPolicy(policy_number)`
- `CREATE INDEX ON :Claim(claim_id)`
- `CREATE INDEX ON :EntityType(name)`
- `CREATE INDEX ON :OntologyClass(name)`

Query Performance:

- **Lookup queries**: $O(1)$ via index on identifiers
- **Traversal queries**: $O(k)$ where k is the number of neighbors (efficient due to Neo4j's native graph storage)
- **Schema queries**: $O(\text{depth})$ for hierarchy traversal (typically depth ≤ 3)

Bulk Operations:

- Use Neo4j APOC library for batch imports (10K nodes per batch)
- Parallelized relationship creation for faster graph construction

3.5.4 Query Interface

Query Types Supported:

1. **Lookup queries**: "Find policy by policy number"
2. **Traversal queries**: "List all claims for policy XYZ"
3. **Pattern matching**: "Find policies with flood claims in zone X"
4. **Aggregation**: "Average claim amount by flood zone"
5. **Multi-hop**: "Find all exclusions involved in denied claims"

Natural Language Interface: A lightweight LLM layer translates natural language questions to Cypher queries. For example:

- User: "Show me flood policies issued in 2024 with claims"
- Cypher: `MATCH (p:Policy)-[:HAS_CLAIM]->(c:Claim) WHERE p.effective_date >= '2024-01-01' RETURN p, c`

4 Datasets

4.1 Track A: NFIP Flood Insurance (Primary)

- **Structured**: FIMA NFIP Redacted Policies v2 and Claims v2 from OpenFEMA
- **Unstructured**: Standard Flood Insurance Policy (SFIP) PDFs—General Property Form (F-123)
- **Size**: ∼100K policy records, ∼50K claims, 3–5 policy form PDFs
- **Key Attributes**: Policy IDs, coverage limits, deductibles, flood zones, claim amounts, dates of loss

4.2 Track B: Auto Insurance (Transfer Study)

- **Structured**: Kaggle Auto Insurance Claims dataset
- **Structured**: Kaggle Vehicle Insurance Customer/Policy dataset
- **Size**: ∼10K claims, ∼50K policies
- **Key Attributes**: Policy IDs, vehicle info, driver info, accident details, claim amounts

4.3 Fusion Objective

1. **Track A**: Link NFIP claims to policy entities and extracted SFIP clauses to form a unified graph showing which coverage terms apply to which claims
2. **Track B**: Apply flood-trained pipeline to auto data; measure how much ontology transfers vs. requires extension
3. **Cross-LOB**: Evaluate schema overlap and extension quality; identify common insurance concepts

5 Evaluation Framework

5.1 Metrics

Table 3: Evaluation Metrics and Targets

Metric	Definition	Target
Schema Precision	% of induced classes matching expert ontology	$> 80\%$
Schema Recall	% of expert classes discovered by system	$> 70\%$
Entity Resolution F1	P/R on entity matching in golden subgraph	> 0.75
Relation F1	P/R on relation extraction accuracy	> 0.70
LOB Transfer Rate	% of flood schema reusable for auto	$> 60\%$
Query Accuracy	Correctness on 20 insurance QA tasks	$> 75\%$

5.2 LOB Transfer Measurement

The key metrics for our contribution measure schema portability:

Primary Metric - Class Transfer Rate (CTR):

$$\text{CTR} = \frac{|\{\text{DIRECT_REUSE}\}| + |\{\text{SUBCLASS}\}|}{|\mathcal{C}_B|} \times 100\% \tag{16}$$

where $\mathcal{C}_B$ is the set of ontology classes induced from LOB B (auto insurance).

Interpretation: CTR measures what percentage of auto insurance ontology classes can reuse (via direct mapping or subclass extension) the flood insurance ontology structure.

Target: CTR $> 60\%$

Secondary Metric - Property Transfer Rate (PTR):

$$\text{PTR} = \frac{\sum_{C_i^B \in \mathcal{C}_B} |P(C_i^B) \cap P(\text{best_match}(C_i^B))|}{\sum_{C_i^B \in \mathcal{C}_B} |P(C_i^B)|} \times 100\% \tag{17}$$

where $P(C)$ is the set of properties for class C.

Interpretation: PTR measures what fraction of properties in auto insurance classes already exist in the matched flood insurance classes.

Target: PTR $> 50\%$

Tertiary Metric - Extension Quality:

For each SUBCLASS or SIBLING extension, we validate correctness:

- Human evaluator reviews 30 randomly sampled extensions
- Classifies each as: Correct / Reasonable / Incorrect
- Compute precision: % of extensions that are Correct or Reasonable

Target: Extension Precision $> 80\%$

Comparison Setup:

We compare two ontologies:

- $\mathcal{O}_{\text{flood}}^{\text{extended}}$: Flood ontology extended with auto data (our approach)
- $\mathcal{O}_{\text{auto}}^{\text{fresh}}$: Ontology induced from auto data alone (baseline)

Both use identical clustering parameters (γ, thresholds, LLM prompts) to ensure fair comparison.

Additional Analysis:

- **Extension type distribution**: Bar chart showing % of each extension type
- **Transfer by semantic distance**: Plot CTR vs. semantic similarity between LOBs
- **Property reuse patterns**: Which property categories transfer best (monetary, temporal, categorical)?

5.3 Baselines

1. **No-Schema Baseline**: Raw LLM extraction without canonicalization or conceptualization
2. **Fixed-Schema Baseline**: Original proposal with manually predefined ontology
3. **GraphRAG**: Microsoft's community-based approach (no schema induction)
4. **Vector RAG**: Standard chunk retrieval with semantic search only

5.4 Golden Subgraph Annotation

We will manually annotate:

- 50 flood insurance cases (policy + claim + relevant SFIP clauses)
- 30 auto insurance cases (policy + claim records)
- Ground truth entity IDs and relationship links
- Expert-validated ontology classes for both domains

Annotation guidelines will be developed iteratively with domain expert review (if available) or based on insurance industry standard taxonomies.

6 Technical Stack

Table 4: Technical Stack (Resource-Constrained)

Component	**Choice**	**Rationale**
Graph Database	Neo4j (AuraDB free / local)	Property graph, Cypher
Extraction LLM	Llama 3.1 8B (Ollama)	Local, no API costs
Verification LLM	DeepSeek-R1 distilled 7B	Strong reasoning, local
Embeddings	all-MiniLM-L6-v2	Fast, good for clustering
Clustering	graspologic (Leiden)	Same as OntoRAG
Orchestration	LangGraph	State machines for agents
PDF Parsing	Unstructured + PyMuPDF	Proven in OntoRAG
UI	Streamlit	Quick demo development

All components run locally or on free tiers, making the project feasible within student resource constraints.

7 Timeline

Table 5: Project Timeline (14 Weeks)

Phase	Weeks	Deliverables
1: Foundation	1	Data acquisition: NFIP, SFIP PDFs, Kaggle auto datasets. Basic EDA.
	2	Zone 1: PDF parsing + CSV ingestion pipeline working. Test on sample data.
	3	Zone 2a: Open IE baseline with Llama 3.1 8B. Evaluate triple quality. Iterate prompts.
	4	Zone 2b-c: Entity/Event classification + entity resolution working. Validate on 100 samples.
		— *Spring Break* —
2: Schema Induction	5	Zone 2c: Entity typing completed. Zone 3a: Type aggregation + similarity graph construction.
	6	Zone 3a: Leiden clustering on types, tune resolution parameter γ. Validate coherence.
	7	Zone 3a: Hierarchy induction + property synthesis. Manual review of top-level classes.
	8	Zone 3b: Validation checks implemented. Zone 3c: Schema extension mechanism designed.
3: LOB Transfer	9	Zone 3c: Extension mechanism implemented and unit tested on toy examples.
	10	Apply pipeline to auto data: Zones 1-2c extraction/resolution/typing (3 days), Zone 3a clustering $\rightarrow \mathcal{O}_B$ (3 days), Zone 3c begin comparison (1 day).
	11	Complete Zone 3c comparisons. Compute CTR, PTR metrics. Iterate on extension logic and tune thresholds.
	12	Comparative evaluation vs. baselines. Run ablation studies (e.g., remove co-occurrence term).
4: Polish	13	Streamlit UI for demo. Natural language query interface. Final evaluation on golden subgraph.
	14	Report writing, code cleanup, presentation slides, practice demo. Buffer for overruns.

Key Changes from Original Timeline:

- Week 5: Split Zone 2c and 3a to avoid overload

- Week 6-7: Separated clustering from hierarchy induction (both are substantial tasks)
- Week 9: Added dedicated week for extension mechanism implementation
- Week 10-11: Two weeks for auto insurance application (expect debugging)
- Week 14: Added explicit buffer for overruns

Risk Contingencies:

- If Weeks 5-8 overrun: Reduce scope of hierarchy induction (limit to 2 levels)
- If Week 10-11 overrun: Use qualitative transfer analysis instead of full quantitative metrics
- If Week 13 overruns: Simplify UI to basic Neo4j Browser instead of custom Streamlit

8 Deliverables

1. **Mid-Term Checkpoint**: Baseline pipeline (Zones 1–2) + initial Neo4j graph on flood data with evaluation on sample data

2. **Final Codebase**: Complete LangGraph implementation of four-zone pipeline with documentation

3. **Knowledge Graphs**: Neo4j exports for (1) flood-only graph ($\mathcal{O}_A$, ∼75K nodes), (2) auto-only graph ($\mathcal{O}_B$, ∼40K nodes), and (3) merged multi-LOB graph ($\mathcal{O}_A^{\text{extended}}$, ∼120K nodes)

4. **Transfer Study Results**: Quantified LOB transfer metrics (CTR, PTR), analysis of extension types, comparison vs. baselines

5. **Demo Application**: Streamlit interface for data upload, ontology visualization, and natural language query

6. **Final Report**: 10–15 pages covering architecture, implementation, evaluation, findings, and discussion

7. **Presentation**: 15-minute slide presentation with live demo

9 Risk Mitigation

Table 6: Risk Assessment and Mitigation Strategies

Risk	**Mitigation**
LLM extraction quality too low	Add few-shot examples from insurance domain; iterate on prompts; consider using GPT-4 for subset validation
Clustering produces nonsense	Human-in-loop review for top-level classes; tune resolution parameter; use hierarchical agglomerative clustering as fallback
LOB transfer fails completely	Pivot to "domain adaptation study" narrative; analyze failure modes systematically; still publish negative results
Time overrun on Zone 3	Phase 3 (LOB transfer) can be scoped to qualitative analysis if quantitative metrics fall short
Neo4j performance issues	Use indexes on key properties; batch imports via APOC; limit graph size to 100K nodes for demo
Data quality issues	NFIP data is well-documented; focus on subset if full dataset is problematic; manual cleaning for Kaggle data

10 Conclusion

This revised proposal presents a feasible, resource-efficient approach to insurance knowledge graph construction that addresses a gap in the literature: **automatic schema evolution for cross-LOB transfer**. By combining proven techniques from OntoRAG (Leiden clustering for ontology induction) and EDC (open extraction + canonicalization), along with AutoSchemaKG's entity/event distinction, we create a streamlined pipeline that induces ontologies directly from extracted instances without requiring predefined schemas or intermediate conceptualization layers.

Our key contribution—the schema extension mechanism with transfer rate measurement—provides a quantifiable way to evaluate ontology portability across insurance domains. This advances both the research understanding and practical applicability of knowledge graphs in the insurance industry, where multi-LOB integration remains a significant challenge.

The project is feasible within the 14-week timeframe using local LLMs and free-tier cloud services. All methods are validated in recent literature and adapted to our specific use case. The streamlined four-zone architecture (Ingestion → Open IE → Schema Evolution → Storage) reduces complexity while maintaining research novelty. The deliverables will demonstrate both technical implementation and research contribution.

References

References

[1] Y. Tiwari, O.A. Lone, and M. Pal, "OntoRAG: Enhancing question-answering through automated ontology derivation from unstructured knowledge bases," *arXiv:2506.00664*, 2025.

[2] J. Bai *et al.*, "AutoSchemaKG: Autonomous knowledge graph construction through dynamic schema induction from web-scale corpora," *arXiv:2505.23628*, 2025.

[3] B. Zhang and H. Soh, "Extract, define, canonicalize: An LLM-based framework for knowledge graph construction," in *Proc. EMNLP*, 2024.

[4] D. Edge *et al.*, "From local to global: A Graph RAG approach to query-focused summarization," *arXiv:2404.16130*, 2024.

[5] Y. Lu *et al.*, "KARMA: Leveraging multi-agent LLMs for automated knowledge graph enrichment," in *Advances in NeurIPS*, 2025.

[6] M.S. Abolhasani *et al.*, "Leveraging LLM for automated ontology extraction and knowledge graph generation," *arXiv:2412.00608*, 2024.

[7] OpenFEMA, "FIMA NFIP redacted policies - v2," `https://www.fema.gov/openfema-data-page/fima-nfip-redacted-policies-v2`.

[8] OpenFEMA, "FIMA NFIP redacted claims - v2," `https://www.fema.gov/openfema-data-page/fima-nfip-redacted-claims-v2`.

[9] FEMA, "NFIP Standard Flood Insurance Policy: General Property Form (F-123)," 2021.

[10] Kaggle, "Auto insurance claims data," `https://www.kaggle.com/datasets/buntyshah/auto-insurance-claims-data`.